RS-DRL: Managing Uncertainty in Self-Adaptive Systems Based on a Novel Continuous Deep Reinforcement Learning Method

ALIREZA KAVIANIFAR, Computer Engineering Department, Tarbiat Modares University, Iran
SAEED JALILI*, Computer Engineering Department, Tarbiat Modares University, Iran

Designing software systems within uncertain and evolving operational contexts is challenging, as pervasive uncertainty can compromise system objectives. Self-adaptive systems address this by dynamically adjusting configurations when objectives are at risk. However, exhaustive exploration of large configuration spaces is time-consuming and computationally expensive. Existing machine learning, search-based, and reinforcement learning approaches often degrade under changing conditions. To overcome these limitations, this paper introduces an architecture based on the MAPE-K loop integrated with a Deep Reinforcement Learning (DRL) module enhanced by a novel reward-shaping mechanism. The proposed Reward-Shaped Deep Reinforcement Learning (RS-DRL) method reshapes rewards during experience replay, improving generalization, convergence, and adaptability across dynamic environments. Experiments on IoT case studies (DeltaIoTv1/v2) show that RS-DRL achieves an asymptotic multi-objective reward of 0.9905 ± 0.0268 compared to 0.4559 ± 0.2233 for the ϵ-greedy DQN baseline, and near-optimal per-objective asymptotic results (e.g., packet loss 0.9850 ± 0.0071, latency 0.9964 ± 0.0107). Under the TT goal, RS-DRL reduces packet loss by up to 22.23% and latency by up to 39.07%, each measured relative to both the best competing method (DLASER+) and the exhaustive-search Reference baseline that analyzes the full adaptation space. All comparisons are based on 30 independent runs, with 25 of 28 baseline comparisons statistically significant (many with $p < 0.001$). These results demonstrate that RS-DRL offers a robust, efficient, and adaptive optimization strategy for self-adaptive systems operating under uncertainty and dynamic conditions.

CCS Concepts: • **Software and its engineering → Software organization and properties**; • **Computing methodologies → Machine learning**.

Additional Key Words and Phrases: Self-adaptive systems, Dynamic Environments, Uncertainty Management, Deep Reinforcement Learning, Reward Shaping

ACM Reference Format:
Alireza Kavianifar and Saeed Jalili. 2026. RS-DRL: Managing Uncertainty in Self-Adaptive Systems Based on a Novel Continuous Deep Reinforcement Learning Method. 1, 1 (January 2026), 31 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Within the domain of contemporary software systems engineering, complexity stands out as a prominent characteristic. The dynamic operational environment assumes a pivotal role in shaping the intricacies of this complexity, compelling systems to grapple with uncertain conditions that often manifest only after they are put into operation. These uncertainties may compromise the objectives of the system. Uncertainty in self-adaptive systems is any deviation

*Corresponding author.

Authors' Contact Information: Alireza Kavianifar, k.alireza@modares.ac.ir, Computer Engineering Department, Tarbiat Modares University, Tehran, Iran; Saeed Jalili, sjalili@modares.ac.ir, Computer Engineering Department, Tarbiat Modares University, Tehran, Iran.

of deterministic knowledge that may reduce the confidence of adaptation decisions made based on the knowledge [47]. For example: 1) systems deployed in dynamic environments, such as IoT applications in smart cities, may face uncertainty in environmental conditions (e.g., weather changes); 2) in cloud environments, uncertainty arises as the demand for resources can fluctuate; 3) in the landscape of cybersecurity, uncertainty manifests itself with new threats emerging continuously, and systems in domains like e-commerce or social media encounter uncertainty in user behavior and preferences; 4) in distributed systems, uncertainties in resource availability can occur due to network latency, hardware failures, or changes in the infrastructure. Effectively addressing these uncertainties ensures that systems remain resilient, responsive, and capable of delivering best performance in diverse and unpredictable scenarios.

Self-adaptation is a prominent and effective approach to deal with uncertainty [29, 46]. Uncertainty in modern software systems—arising from unpredictable environmental changes, fluctuating resource availability, or evolving user goals—can significantly impair a system's ability to meet its objectives. Traditional static or rule-based systems lack the flexibility to respond to such runtime variability. In contrast, self-adaptive systems (SASs) are equipped with feedback loops that enable them to monitor the environment and the system's own state, analyze deviations from expected behavior, and autonomously plan and enact adaptations. These runtime capabilities allow an SAS to dynamically modify its structure and behavior to maintain desired system qualities such as reliability, performance, or efficiency. Since uncertainty often emerges at runtime, design-time solutions alone are insufficient to guarantee system robustness. As a result, self-adaptation has become a foundational paradigm for managing uncertainty across various domains, including robotics [15], Internet of Things [49], cyber-physical systems [36], self-driving vehicles [21], and smart homes [7]. This growing importance of self-adaptation underpins the need for techniques—such as the one proposed in this work—that enhance adaptability in the face of dynamic and uncertain environments.

Within this paper, our focus is on addressing self-adaptive software systems featuring large discrete adaptation spaces. The adaptation space denotes the entirety of potential configurations achievable from the current state of the system by applying a defined set of adaptation actions. Each configuration is treated as an adaptation option, and the size of the adaptation space may remain constant or undergo dynamic changes.

When confronted with an extensive adaptation space, the task of selecting an adaptation option that aligns with system objectives becomes computationally demanding and, in certain instances, nearly unattainable [13, 48]. A prevalent approach for selecting the suitable adaptation option involves the validation of runtime models associated with both the system and its environment, focusing on one or more quality properties. These quality models encompass parameters that can be configured to depict a specific adaptation option [4, 34, 49]. However, the verification of these runtime models is a time-consuming endeavor, particularly in software systems where swift responses to changes are imperative, making the current methodology impractical. Consequently, the quest for more efficient solutions is imperative to assist in the identification of suitable adaptation options within vast adaptation spaces for time-constrained software systems.

Existing works and gap. *Finding suitable adaptation option.* Several methods have been proposed to find suitable adaptation option including Machine learning-based [23, 39, 40, 45, 50] and search-based methods [8, 10, 26–28, 38].However, Collecting data for optimization can be expensive or infeasible. Additionally, as the environment changes, the performance of the model learned by these methods degrades and is no longer able to satisfy the performance requirements of the software system.

Online Reinforcement Learning (RL) has emerged as a promising technique for optimizing software systems, particularly those operating in dynamic and uncertain environments [31]. Unlike traditional optimization methods, which rely on pre-defined rules or static models, RL employs a more adaptive approach, allowing systems to learn from

their interactions with the environment in real time. This capability makes RL particularly well-suited for applications where system performance needs to be continuously optimized in response to changing conditions. For example, in a cloud computing environment, an RL agent might learn to allocate resources dynamically based on the observed demand, thereby reducing costs while maintaining performance. As the environment evolves—perhaps due to a sudden spike in user demand—the RL agent adjusts its strategy in real-time, using the newly collected data to refine its model.

Despite the potential of RL, particularly in its deep learning variant (Deep Reinforcement Learning or DRL), there are significant challenges that must be addressed. One of the most critical issues is the problem of local optima [14]. Local optima refer to solutions that are optimal within a limited region of the problem space but are not the best possible solutions globally. In the context of DRL, this means that the model may learn a policy that performs well under certain conditions but fails to generalize when the environment changes. This issue is particularly problematic in dynamic environments where the conditions are not static and can vary significantly over time. For instance, a DRL model trained to optimize packet loss in a smart city's IoT network might perform well when the network traffic is stable and predictable. However, if there is a sudden surge in network traffic—such as during a citywide event or emergency—the model might struggle to adapt. It could continue to use the same routing strategies that worked previously, even though they are no longer effective, leading to increased packet loss and potentially critical communication failures.

The inability of DRL models to generalize well to unseen environments is a significant limitation. Unlike traditional machine learning models that can be retrained periodically on new data, DRL models are often deeply integrated into the system's operation. This integration means that retraining the model is not just time-consuming but can also disrupt the system's performance. Retraining involves several steps: collecting new data, re-evaluating the model's structure and parameters, and testing the updated model to ensure it performs better than the previous version. During this process, the system might operate suboptimally, as the old model is no longer suitable, but the new model is not yet fully trained. In critical systems, such as those controlling autonomous vehicles or managing financial transactions, this period of suboptimal performance can have serious consequences. Moreover, in rapidly changing environments, the retraining process might need to be repeated frequently, leading to a cycle of continuous adaptation that consumes significant computational resources and human oversight [9, 52].

To conclude, the dynamic and unpredictable nature of environments in which software systems operate poses significant challenges for traditional Deep Reinforcement Learning (DRL) models. These models often struggle with local optima and cannot generalize well when faced with changing conditions, leading to suboptimal performance and the need for frequent, time-consuming retraining [37].

Our method: Reward-Shaped Deep Reinforcement Learning (RS-DRL). To address these challenges, we propose a novel DRL-based method, *Reward-Shaped Deep Reinforcement Learning (RS-DRL)*, aiming to autonomously manage environmental changes through an efficient and effective retraining process.

To that end, we aim to use architecture-based adaptation [17], which adds an external feedback loop to the system. The proposed architecture consists of two main systems: the *Managed System* and the *Managing System*. The Managed System is the software system that needs adaptation, The Managing System sits on top of the Managed System and operates based on a feedback loop that monitors, analyzes, plans, and executes adaptation options. We extend the Managing System by adding a DRL-based module to it. So its architecture is consisted of two primary modules: the *MAPE-K Loop* and the *RS-DRL Module*, working together to manage uncertainty in dynamic environments.

The *MAPE-K Loop* (Monitor, Analyze, Plan, Execute, Knowledge) interacts with the Managed System by sensing relevant data, analyzing it, and predicting suitable adaptation option using the trained RL model. This loop manages the system by executing the chosen adaptation option and adjusting behavior in response to environmental changes.

The *RS-DRL Module* is responsible for training a model that learns the suitable adaptation option through reinforcement learning. A reward mechanism drives this learning process, allowing the model to continuously evolve and improve decision-making over time. By integrating these components, the architecture enables the system to autonomously adapt to dynamic environments, ensuring enhanced performance and efficient resource management.

Novel Reward Shaping Mechanism. The novel aspect of RS-DRL lies in the innovative use of reward shaping during the experience replay phase of the Deep Reinforcement Learning (DRL) training process. Reward shaping is a well-known technique that modifies the reward function to guide the learning process more effectively. However, RS-DRL introduces a unique twist inspired by how humans learn from their experiences, particularly from failures.

In human learning, we often reflect on our failed attempts, analyze what went wrong, and use those lessons to improve our future performance. This process not only helps us avoid repeating the same mistakes but also allows us to anticipate challenges and adapt more quickly to new situations. We have embedded this concept into our DRL framework.

In traditional DRL, experiences are collected during the agent's interaction with the environment, and these experiences are replayed during training to reinforce learning. The rewards associated with these experiences are typically reflective of the actual outcomes—positive rewards for successful actions and negative rewards for failures. However, this standard approach can limit the agent's ability to generalize to new, unseen environments and can contribute to the problem of local optima, where the agent learns strategies that work well in specific situations but fail to adapt when conditions change.

To address the limitations of traditional DRL, RS-DRL introduces a reward reshaping mechanism inspired by human learning. This approach encourages the agent to reframe failed experiences as if they had succeeded, transforming failures into valuable lessons that accelerate learning, much like human adaptive behavior. The reshaped reward structure helps the model generalize better to unseen environments by promoting the exploration of a wider range of states and actions during experience replay, reducing the risk of getting stuck in local optima. By encouraging the agent to explore strategies beyond those initially perceived as suboptimal, RS-DRL prevents the agent from being trapped in local optima, which is a common challenge in traditional DRL when conditions change. This broader exploration leads to the discovery of more effective strategies in dynamic environments. As a result, RS-DRL produces more robust models that adapt swiftly to new conditions, improving the efficiency of the learning process and maintaining optimal performance without the need for extensive retraining.

Contributions: The main contributions of RS-DRL are summarized as follows:

(1) *Novel reward-shaping mechanism for generalization.* We propose **randomized reward reshaping** (Algorithm 2), a strategy that treats failed experiences as successful during experience replay. Unlike goal-conditioned methods (e.g., HER), it requires no predefined goals or domain knowledge, applies a constant reward ($r_i \leftarrow 1$) to simulate optimistic outcomes, and promotes exploration that helps avoid local optima traps (Section 5.2.1).
(2) *Integration of RS-DRL within the MAPE-K loop.* We design a self-adaptive framework where the MAPE-K loop detects deviations from expected behavior and triggers retraining only when performance thresholds are violated (Section 4.1.1). This tight integration enables efficient, on-demand learning, achieving 22.23% faster adaptation than DLASER+ [50] in IoT network scenarios (Table 5).
(3) *Scalability and effectiveness in large adaptation spaces.* RS-DRL is validated on multiple instances of the DeltaIoT application (up to 4,096 configurations), demonstrating substantial improvements in system metrics (e.g., 39.07% lower latency in DeltaIoTv2) and supporting efficient adaptation in large, dynamic environments (Section 6.5).

2 Related Work

In recent years, there has been growing interest in employing autonomous techniques to find suitable adaptation options, particularly in software systems with large adaptation spaces. We classify these methods into three main categories: Classic and deep machine learning methods [23, 39, 40, 45, 50], search-based methods [8, 10, 26–28, 38], and classic and deep reinforcement learning methods [2, 3, 5, 6, 25, 31, 35, 42, 53].

2.1 Classic and deep machine learning-based methods

Authors in [39, 40] present a machine learning method to reduce large adaptation spaces. Their method enhances the traditional MAPE-K feedback loop with a learning module that selects subsets of adaptation options from a large adaptation space to support the analyzer with performing efficient analysis. It is instantiated for two concrete learning techniques, classification and regression, and evaluated for two instances of an Internet of Things application for smart environment monitoring with different sizes of adaptation spaces. The evaluation shows that both learning methods reduce the adaptation space significantly without noticeable effect on realizing the adaptation goals.

Authors in [23, 50] propose a deep-learning-based method. First, a deep learner is applied to reduce the adaptation space for the threshold goals and then ranks these options for the optimization goal; the adaptation options are verified one by one with respect to their predicted ranking until an option that complies with the threshold goals is encountered.

The authors in [22] proposed a method where Pareto optimal configurations are learned offline and used at runtime to generate adaptation plans, reducing adaptation spaces. Model checking with PRISM [30] is applied at runtime for quantitative reasoning. Evaluated in robot missions focused on task timeliness and energy consumption, the method shows promising results, selecting high-quality adaptation plans in uncertain environments. However, its effectiveness depends on the suitability of the machine learning model to new conditions, and limiting the search to Pareto-optimal configurations may exclude other beneficial options.

In conclusion, Classic and deep machine learning-based methods require labeled training data representative of the system's environment, which may be challenging to obtain due to design time uncertainty.

2.2 Search-based methods

Authors in [10] highlighted the need for improved planning techniques in self-adaptive systems, optimized for multiple evolving and hard-to-measure quality attributes. They advocate for the application of stochastic search techniques, such as hill climbing and genetic algorithms, to navigate complex, multi-dimensional search spaces. Recent advances in these fields make this approach particularly promising. To demonstrate its feasibility, the authors apply a search-based planning prototype, offering potential research directions. In the long term, this technique could enhance domain knowledge, reduce human effort, and increase the utility of self-adaptive systems.

Authors in [26–28] proposed a planner based on genetic programming that reuses existing plans. Their method uses stochastic search to deal with unexpected adaptation strategies, specifically by reusing or building upon prior knowledge. Their genetic programming planner is able to handle very large search spaces. However, it is important to note that although the method of reusing prior planning knowledge to adapt to unexpected situations can be beneficial, it also has potential limitations. For example, the effectiveness of the method can be influenced by the quality and relevance of existing plans that are reused. If these plans do not adapt well to new conditions, their reuse may not lead to optimal adaptation. Furthermore, the authors point out that naively reusing existing plans for self-adaptation

can actually lead to a loss of utility. This suggests that further investigation and potentially additional techniques are needed to ensure that plan reuse is beneficial.

Authors in [8] introduced FEMOSAA, a framework that integrates feature models with Multi-Objective Evolutionary Algorithms (MOEA) to optimize Self-Adaptive Software (SAS) at runtime. FEMOSAA operates in two phases: design time, where the SAS feature model is adapted to the MOEA, and runtime, where the model guides the search for better solutions. Tested on two SAS systems, including a complex real-world application, FEMOSAA showed superior performance compared to other frameworks. However, its effectiveness depends on the suitability of the feature model and MOEA, and focusing solely on knee solutions may exclude other beneficial configurations.

Authors in [38] used a genetic algorithm to automatically generate adaptation plans to adapt a system along with reconfiguration schemes. The generated plans are optimal in terms of performance considering the available resources (eg, battery). Specifically, plans are defined as modifications of the application software architecture based on the so-called feature model. They used a case study involving an application that assists participants in international congresses, keeps them up-to-date with the latest news, and provides several social amenities.

2.3 Classic and deep reinforcement learning-based methods

Authors in [25] proposed a reinforcement learning-based method with epsilon-greedy action selection for online planning in self-adaptive systems. This approach enables software systems to learn from outcomes, balancing exploration and exploitation to adapt effectively to environmental changes. A robot battle simulator case study demonstrates that this epsilon-greedy RL approach enhances architecture-based self-adaptation.

Authors in [5] proposed a method called RLMC (Reinforcement Learning and Model Checker) for architecting Internet of Things (IoT) systems that can guarantee Quality of Service (QoS) levels. Their method combines RL (with epsilon-greedy) and formal quantitative verification (probabilistic model checking). In this method, RL is tasked with selecting the best adaptation option, and quantitative verification checks the feasibility of the adaptation decision. This avoids the execution of infeasible adaptations and provides feedback to the ML engine that helps achieve faster convergence towards optimal decisions. Their evaluation results show that their method can produce better decisions than ML and quantitative verification used in isolation.

Authors in [31] used an RL-based method to address the problem of exploration-exploitation problem in Self-adaptive software systems. In their exploration strategies they use feature models to organize the system's adaptation space and thereby leverage additional information to guide exploration. In addition, their strategies detect added and removed adaptation actions by analyzing the differences between the feature models of the system before and after an evolution step. Adaptation actions removed as a result of evolution are no longer explored, while added adaptation actions are explored first.

Authors in [6] proposed a self-organizing fully decentralized solution for the service assembly problem. The aim of ensuring good overall quality for the services provided is to ensure fairness among participating peers. The main features of this solution are: (1) The use of a gossip protocol to support decentralized information dissemination and decision making. (2) The use of a reinforcement learning method to make each peer able to learn from its experience the service selection rule to be followed, thus overcoming the lack of global knowledge. Additionally, they explicitly take into account load-dependent quality attributes, which lead to the definition of a service selection rule that drives the system away from overloading conditions that could adversely affect quality and fairness. Simulation experiments show that their solution adapts to changes by rapidly converging to viable sets while maintaining the specified quality and fairness objectives.

Authors in [3] proposed using Q-learning with an ϵ-greedy strategy to address the challenges of determining optimal scaling policies in cloud services like Amazon. These services use virtualization to run multiple virtual machines on a single physical server, often leading to performance interference between co-located machines. This makes scaling decisions in dynamic environments difficult. To mitigate the "curse of dimensionality" in reinforcement learning, the authors introduced a parallel Q-learning approach to accelerate the discovery of optimal policies in real-time.

Authors in [2] compared two dynamic learning strategies based on a fuzzy logic system. The key problem addressed is how and when to add/remove resources in order to meet agreed service-level agreements. Reducing application cost and guaranteeing service-level agreements (SLAs) are two critical factors of dynamic controller design. The authors proposed a self-adaptive fuzzy logic controller combined with two reinforcement learning (RL) methods: (1) Fuzzy SARSA learning (FSL). (2) Fuzzy Q-learning (FQL). Both methods are implemented and compared in their advantages and disadvantages in the OpenStack cloud platform. The authors demonstrate that both auto-scaling methods can handle various load traffic situations, sudden and periodic, and deliver resources on demand while reducing operating costs and preventing SLA violations.

Authors in [35] proposed two novel algorithms that aim to achieve greater data efficiency by saving experience data and using it in aggregate to make updates to the learned policy. The first algorithm introduces an offline learning scheme for service composition where the online learning task is transformed into a series of supervised learning steps. This method is designed to enhance the efficiency of reinforcement learning algorithms, which are commonly used to compose and adapt Web services in open and dynamic environments. However, these algorithms are often relatively inefficient in their use of experience data, which may affect the stability of the learning process. By saving experience data and using it in aggregate to make updates to the learned policy, the proposed algorithms aim to overcome this limitation and improve the efficiency of the learning process.

The authors in [53] examined self-adaptive systems' challenges and the need for runtime adaptations as environments or user goals change. They highlighted limitations in rule-based adaptation, such as fixed rules that may not be optimal or flexible. To address this, they proposed a RL framework (with epsilon-greedy) that generates and evolves adaptation rules, learning offline from goal settings and adapting online with real-time data. Applied to an E-commerce web application, the framework improved self-adaptation efficiency and effectiveness.

Authors in [42] discussed the application of reinforcement learning to automate the process of energy-efficient virtual machine consolidation in cloud data centers. The authors proposed a reinforcement learning-based method to automate this process. This method enables the system to learn from its experience and dynamically adjust its actions based on this learning.

In conclusion, none of the methods reviewed above address the problem of large adaptation spaces. The main problem with them is that all of them use classic reinforcement learning methods such as Q-Learning or Sarsa. These classic RL methods cannot handle large adaptation spaces and are suitable for problems where the size of adaptation space is small.

3 Motivating Example

In this section, we motivate our method by illustrating the necessity of having a more effective optimization strategy for software systems operating under changing environmental conditions.

DeltaIoT [20] , an Internet-of-Things application developed by VersaSense, exemplifies the complexities and challenges of managing IoT networks in dynamic and unpredictable environments (Figure 1). This system comprises a set of sensors, each equipped with battery-powered motes responsible for routing sensor data through a wireless multi-hop

Fig. 1. DeltaIoTv1 [20]

communication network to a gateway. The gateway then connects this data to a user application, where it can be analyzed and utilized for various purposes, such as monitoring environmental conditions or managing industrial processes.

DeltaIoT can be configured with varying numbers of motes and adaptation options, depending on the specific requirements of the study. In this article, we consider three specific instances of DeltaIoT. The first instance, *DeltaIoTv1*, comprises 15 motes and 216 adaptation options. The second instance, *DeltaIoTv1.1*, introduces an additional mote, increasing the number of adaptation options to 1,296, and the third instance consists of 37 motes and 4096 adaptation options. The number of adaptation options in each instance is determined by network parameters such as power settings and link distribution, which govern how the system can be adapted.

In the DeltaIoT system, the primary objectives are to ensure reliable communication, minimize packet loss, and extend the operational life of motes by optimizing energy consumption. However, achieving these goals is far from straightforward due to the uncertainties that affect the network's performance. The uncertainty in DeltaIoT stems from two main sources:

(1) Network Interference and Noise: DeltaIoT operates in a wireless environment where the quality of communication is heavily influenced by external factors. For instance, network interference can arise from environmental changes, such as atmospheric conditions, or from human activities, like ongoing maintenance work in the vicinity. This interference, which can vary widely from -40 dB to +15 dB, introduces noise into the communication channels, making it challenging to maintain a stable and reliable network.

(2) Variability in Message Load: The sensors in DeltaIoT have different data transmission patterns. Some sensors, like RFID sensors, send data sporadically based on the availability of new information, while others, such as temperature sensors, transmit data at regular intervals. This variability in message load, which can range from 0 to 10 messages per mote, adds another layer of complexity to managing the network. Fluctuations in the volume

of data being transmitted can lead to congestion, increased packet loss, and higher energy consumption as motes struggle to maintain optimal performance under changing conditions.

3.1 The Impact of Data Shifts on Network Performance

To further illustrate these challenges, consider Figure 2, which depicts the relationship between packet loss and energy consumption across various adaptation cycles in the DeltaIoT network. Here each adaptation cycle can be considered a change in environment.

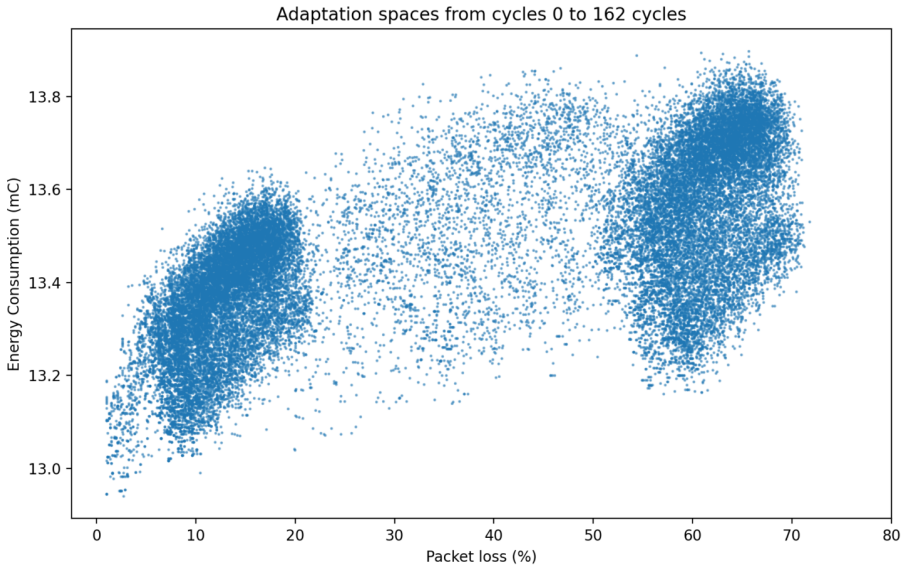


Fig. 2. Adaptation spaces from cycles 0 to 162 showing the impact of data shifts on packet loss and energy consumption (adaptation-cycle traces derived from Gheibi and Weyns [18]).

As shown in Figure 2, the data points indicate how shifts in network conditions, such as increased interference or variations in message load, can lead to significant changes in packet loss and energy consumption. Initially, when the network operates under relatively stable conditions, packet loss remains low, and energy consumption is optimized (left cluster of data points). However, as the network experiences shifts—such as increased interference or a sudden surge in message load—the system's performance degrades, leading to higher packet loss and increased energy consumption (right cluster of data points).

3.2 The Need for an Adaptive and Efficient Optimization Strategy

Given the dynamic nature of the DeltaIoT environment, a static approach to network management is insufficient. Traditional methods, which rely on fixed routing strategies and pre-defined rules, often fail to adapt to the rapidly changing conditions, leading to suboptimal performance. For instance, if a routing path that was initially efficient becomes congested due to increased message load or deteriorates due to heightened interference, the system must be able to detect this change and adjust accordingly to avoid packet loss and excessive energy use.

The unpredictable fluctuations in network interference and message load necessitate a more adaptive approach that can dynamically respond to these changes. Under these conditions, the traditional DRL model, which was trained under the assumption of relatively stable interference and moderate message loads, starts to struggle. Packet loss increases, and some motes begin to drain their batteries faster as they attempt to compensate by boosting their transmission power or repeatedly retransmitting lost packets. This is where our proposed method, *RS-DRL*, which integrates a novel reward shaping mechanism into the Deep Reinforcement Learning (DRL) process, becomes crucial.

3.3 RS-DRL: Ensuring Robustness and Adaptability

In this scenario, RS-DRL shines by addressing these challenges through a combination of real-time change detection and adaptive learning. The self-adaptive architecture we propose continuously monitors the network's performance metrics, such as packet loss rates and energy consumption. As soon as it detects increased interference and message load, it triggers the DRL agent to reassess its strategies.

The novel reward shaping mechanism of RS-DRL becomes critical at this stage of adaptation. The agent revisits past experiences where similar failures occurred—such as increased packet loss due to interference—and reshapes the rewards associated with those experiences. By treating some of these failed experiences as successful under the new conditions, the agent learns to generalize its strategies, allowing it to quickly adapt to the current environment.

For example, the agent might learn that a routing path previously deemed inefficient due to lower interference levels is now the best option under higher interference conditions. Similarly, it might adjust the power settings of motes dynamically, lowering them during periods of low traffic to save energy and increasing them only when absolutely necessary to maintain communication quality.

4 Background

Our method uses a DRL-based method that leverages runtime models of the system and environment to receive feedback from the system and learns based on them. In this section, the necessary background for this article is introduced. First, we introduce MAPE-K loop and DRL and then we explain how the analysis of selected adaptation options are performed using statistical model checking.

4.1 MAPE-K Loop

A pivotal development in the realm of self-adaptive systems is represented by IBM's conceptual reference model, originally termed the Autonomic manager [24], now recognized as the Managing System (Figure 3). Positioned atop the managed system, the Managing System's primary functionality is autonomous adaptation of the managed system. The model comprises distinct functions, elucidated as follows: 1) Knowledge: Serving as the foundational function, Knowledge is shared among all components, encapsulating a representation of the underlying system, its context, and the objectives to be achieved. 2) Monitor dynamically acquires data from the managed system and its contextual environment during runtime, ensuring that the Knowledge remains contemporaneous and synchronized with the state of the managed system. 3) Drawing insights from the Knowledge, the Analyzer assesses the necessity for adaptation. If adaptation is deemed necessary, the Analyzer meticulously scrutinizes potential adaptation options. 4) In instances where adaptation is warranted, the Plan function formulates a meticulous plan, outlining the requisite actions to reinstate the system to a desired state aligned with its objectives. 5) Finally, the Execute element enacts the plan adapting the managed element.

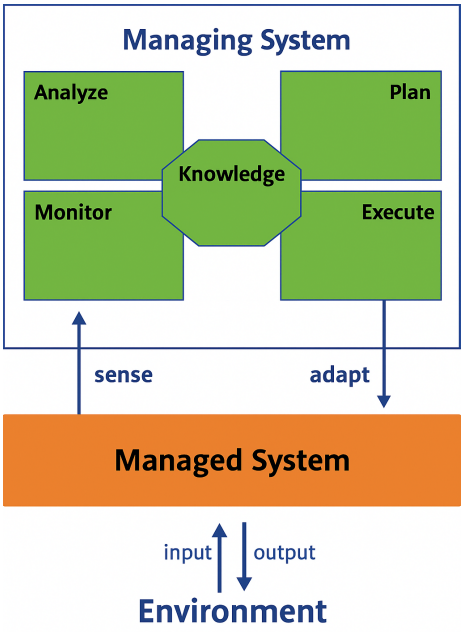


Fig. 3. Conceptual model of a self-adaptive system.

4.1.1 Operational Assumptions. In this study, we make the following assumption about the MAPE-K loop's functionality:

- We assume that the self-adaptive system uses a robust change detection mechanism within the analysis phase. This mechanism can effectively detect any significant deviations from expected performance metrics and notify the plan to initiate a new adaptation option.

This assumption is critical to ensuring the effectiveness of the proposed reward shaping method, as it provides the basis for timely and accurate adaptation and retraining within the self-adaptive system.

4.2 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) is a subfield of machine learning that combines deep learning techniques with reinforcement learning algorithms [32]. Reinforcement learning is a type of machine learning where an agent learns to make decisions by interacting with an environment and receiving feedback in the form of rewards or penalties. The goal of the agent is to learn a policy that maximizes the cumulative reward over time [33].

DRL extends traditional reinforcement learning by employing deep neural networks to approximate the value function or policy function $Q(s, a; \theta_i)$. Here, θ_i represents the parameters (i.e., weights) of the Q-network at iteration i.

To facilitate experience replay, the agent's experiences are stored sequentially at each timestep t in a dataset $D_t = (e_1, \ldots, e_t)$, where each experience $e_t = (s_t, a_t, r_t, s_{t+1})$. During the learning process, Q-Learning updates are applied using samples (or minibatches) of experience (s, a, r, s') drawn uniformly at random from the pool of stored

samples D. The Q-learning update at iteration i uses the following loss function:

$$\mathcal{L}(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim U(D)}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)\right)^2\right], \tag{1}$$

where γ is the discount factor, $\mathbb{E}_{(s,a,r,s')\sim U(D)}$ denotes the expectation over the uniform distribution of samples in D, and $\max_{a'} Q(s', a'; \theta_{i-1})$ represents the maximum Q-value for the next state s' using the previous iteration's parameters θ_{i-1}.

4.3 Analysis of adaptation options

The model of the system and environment are designed as networks of timed automata using UPPAAL-SMC [11]. UPPAAL-SMC allows for the efficient analysis of performance properties of networks of timed automata under a natural stochastic semantics. At runtime, the models, intricately constructed as networks of timed automata, are executed directly using the ActivFORMS execution engine [51]. This dynamic execution allows for the real-time simulation and observation of the modeled system's behavior based on the parameters and conditions specified in the timed automata. ActivFORMS serves as the execution platform, enabling a direct translation of the abstract models into tangible, executable instances, facilitating a practical exploration of the system's responses to various stimuli and conditions. ActivFORMS guides the adaptation of the system at runtime by analyzing selected adaptation options to assess the realization of adaptation goals with a required level of accuracy and confidence in an efficient manner using statistical model verification techniques at runtime.

In essence, this integrated method, involving the use of UPPAAL-SMC for stochastic analysis and the ActivFORMS execution engine for runtime simulation, establishes a comprehensive methodology for both modeling and analyzing complex systems. The synergy between these tools provides a means to not only assess the system's performance properties but also to observe and understand its behavior in real-time scenarios, contributing to a holistic understanding of the system's dynamics and capabilities.

5 Proposed Method

Figure 4 illustrates the integration of RS-DRL with a self-adaptive system, highlighting the use of Deep Reinforcement Learning (DRL) to predict suitable adaptation options. The diagram presents a high-level overview of the main components and the interactions among them. Below is a detailed description of the responsibilities assigned to each component within the self-adaptive architecture:

Managed System. Takes inputs from its environment and produce outputs aligned with user-defined goals. It constantly monitors its state, including system properties, uncertainties, and deviations from desired behavior.

Managing System. Oversees the operation of the managed system, employing a MAPE-K loop and a RS-DRL module to facilitate adaptations.

MAPE-K loop. Consists of several components; the Monitor, which continuously observes the managed system and its environment, collecting relevant data; the Analyzer, which processes the collected data to identify potential issues or deviations from adaptation goals; the Planner, which activates the Adaptation Option Predictor when issues are detected; the Adaptation Option Predictor, utilizing a trained DRL model to predict suitable adaptation options based on the system's current state; the Executor, which retrieves detailed information from the Knowledge Repository and implements the chosen adaptation option, modifying the managed system's behavior or configuration accordingly.

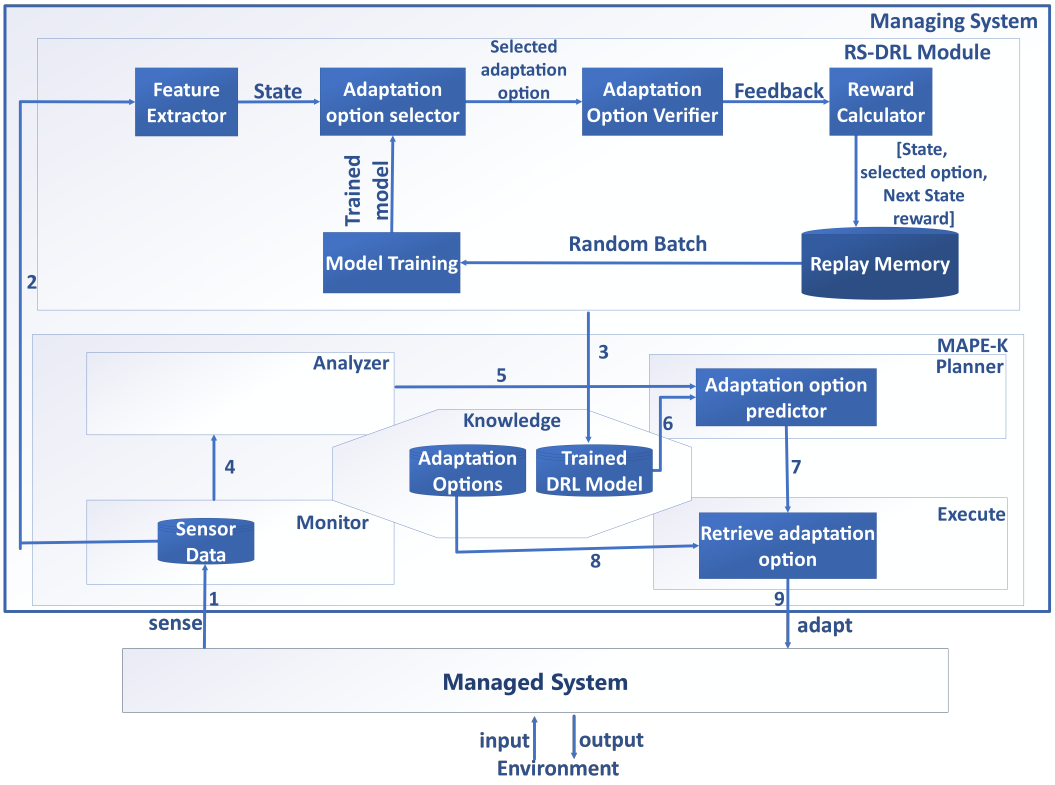


Fig. 4. Integration of RS-DRL with Self-adaptive system

RS-DRL Module is integrated as an upper layer above the MAPE-K loop. It manages the continual refinement of the DRL model and updates the trained model stored in the Knowledge repository. The training process is performed for a certain number of episodes. At the beginning of each training episode, the monitor component of the MAPE-K collects required runtime data (1) and sends them to the RS-DRL module (2). Then for the duration of the training, the following steps are done. Based on received input vectors, the RS-DRL module starts the training process. At the end of the training process, the trained model is stored in the knowledge repository of the MAPE-K loop (3). The Monitor component forwards the collected runtime data to the Analyzer for further evaluation (4). When the Analyzer detects that the adaptation goals are violated or may no longer be achievable, it notifies the Adaptation Option Predictor component embedded in the planner (5). The Adaptation Option Predictor retrieves the model (6) and predicts an adaptation option (7). The Executor retrieves the predicted adaptation option from the adaptation options repository (8) and applies it to the managed system (9), and the whole process of training will be repeated for the next adaptation episode. We now explain the responsibility of key components involved in training RS-DRL model in more detail:

5.1 Key components and responsibilities

Sensor Data collects raw data from the system and environment, such as sensor readings and system logs.

Feature Extractor processes the raw data, producing a state representation of the system that reflects key performance metrics.

Adaptation Option Selector identifies potential adaptation options using an Epsilon-greedy policy, balancing exploration of new strategies with exploitation of known optimal actions. To predict the best option, the Selector uses a deep Q-network (DQN) [32], which estimates Q-values for each action based on the current system state.

Adaptation Option Verifier simulates the impact of each option using runtime models and statistical model checking (Uppaal-SMC [11]) to estimate quality properties like energy consumption, packet loss, and latency.

Reward Calculator assigns a reward based on the Verifier's estimates, guiding the RS-DRL learning process.

Replay Memory introduces a novel reward-shaping mechanism. Unlike traditional experience replay, RS-DRL reshapes certain failed experiences, treating them as successful to mimic human learning. This mechanism: 1) Encourages generalization across diverse environments, 2) Accelerates learning by transforming past mistakes into valuable lessons.

Model Training involves sampling a batch of experiences from the replay memory and updating the DQN using backpropagation to minimize the loss function. The network's weights are adjusted iteratively to better approximate Q-values for future decisions.

Trained Models Repository stores the latest RS-DRL model, which is used in subsequent adaptation episodes.

5.2 Novel Reward Shaping Mechanism in RS-DRL

We now delve into the main novelty of RS-DRL, specifically the reward shaping mechanism that helps the model avoid local optima and find the suitable adaptation option in an effective and efficient manner.

We used an enhanced version of the Deep Q-Network (DQN) algorithm for the implementation of RS-DRL. DQN offers significant advantages over other DRL algorithms for handling large adaptation spaces, particularly in environments with discrete action options, like software configuration tuning. It excels in discrete action spaces by efficiently evaluating each option and is highly sample-efficient due to its use of experience replay, which allows reusing past interactions to reduce data requirements. DQN also benefits from stability through target networks, reducing the risk of instability during training. As an off-policy algorithm, it can learn from historical data, unlike on-policy methods like PPO, which require more real-time interactions. DQN's epsilon-greedy exploration is simpler to implement than the more complex exploration mechanisms in other algorithms, and it has lower computational complexity, making it easier to scale to larger state-action spaces. These features make DQN particularly suitable for managing large, discrete adaptation spaces in software systems [12].

Algorithm 1 presents the RS-DRL algorithm. The algorithm aims to train an action-value function $Q(s, a)$, which represents the learned knowledge, i.e., the expected cumulative reward when performing an action a in a state s. The algorithm includes several important hyperparameters: the learning rate η, which controls how much newly acquired knowledge overrides old information, and the discount factor γ, which dictates the relevance of future rewards. Another parameter, the exploration rate ϵ, determines the balance between exploration (trying new actions) and exploitation (choosing the best-known actions).

After initializing the action-value function Q and its target counterpart $\hat{Q}$ with random weights (lines 1–2), the algorithm proceeds by running through multiple episodes (line 3). At the start of each episode, the environment is initialized, and the initial state s_0 is obtained (line 4). For each step in the episode, the algorithm selects an action a_t

using an ϵ-greedy policy based on the current state-action value function (line 6). The selected action is then executed, and the resulting reward r_t and next state s_{t+1} are observed (line 7).

The experience (s_t, a_t, r_t, s_{t+1}) is stored in a replay buffer D (line 8). From this buffer, a random minibatch B of experiences is sampled (line 9), where $B = \{(s_i, a_i, r_i, s_{i+1})\}_{i=1}^{N}$. Let $\rho \in [0, 1]$ denote the reshaping factor that controls the proportion of minibatch elements to modify. We form a random index set $\mathcal{K} \subseteq \{1, \ldots, N\}$ with $|\mathcal{K}| = \lfloor \rho N \rfloor$. In practice (see Algorithm 2), selection is restricted to experiences that correspond to *failures* (i.e., transitions with $r_i = 0$ under our binary reward assumption). If fewer than $\lfloor \rho N \rfloor$ failures exist, all failures are reshaped. For each $i \in \mathcal{K}$, the reward is replaced by a constant optimistic value of 1, while all other rewards remain unchanged:

$$\tilde{r}_i = \begin{cases} 1, & \text{if } i \in \mathcal{K}, \\ r_i, & \text{otherwise.} \end{cases} \tag{2}$$

This reshaping is applied *only during experience replay*: the modified rewards $\tilde{r}_i$ are used to compute the temporal-difference (TD) targets in Step 12 of Algorithm 1, while the original transition rewards stored in the replay buffer remain unchanged. The intuition is that assigning an optimistic constant reward to a subset of failed experiences encourages the agent to revisit and explore previously unsuccessful action paths. The choice of a constant value of 1 is deliberate and domain-agnostic: since many adaptation goals in self-adaptive systems are naturally binary (goal satisfied / not satisfied), this reshaping injects a hypothetical success signal that promotes exploration without requiring any domain-specific reward engineering.

Input: Replay buffer D of capacity N, learning rate η, reward shaping factor ρ, discount factor γ, exploration rate ϵ

Output: Trained action-value function Q

1. Initialize action-value function Q with random weights θ;
2. Initialize target action-value function $\hat{Q}$ with weights $\hat{\theta} \leftarrow \theta$;
3. **for** *each episode* **do**
    4. Initialize environment and get initial state s_0;
    5. **for** *each step in the episode* **do**
        6. Select action a_t using ϵ-greedy policy based on $Q(s_t, a_t)$;
        7. Execute action a_t and observe reward r_t and next state s_{t+1};
        8. Store experience (s_t, a_t, r_t, s_{t+1}) in replay buffer D;
        9. Sample random minibatch B of experiences (s_i, a_i, r_i, s_{i+1}) from D;
        10. `// Reward Shaping`
        11. RewardShaping(B, ρ);
        12. Compute target y_i for each minibatch transition using reshaped rewards from Equation (2);
        13. Perform a gradient descent step on $(y_i - Q(s_i, a_i; \theta))^2$ with respect to the network parameters θ, using learning rate η;
        14. Update target network parameters $\hat{\theta} \leftarrow \theta$ periodically;
        15. Decrease exploration rate ϵ according to decay strategy;
    end
end

Algorithm 1: RS-DRL

5.2.1 Reward Shaping Function. Algorithm 2 defines the reward shaping function used in the RS-DRL algorithm (Algorithm 1). The purpose of this function is to modify the rewards of a subset of experiences within each minibatch to guide the agent's learning in a way that promotes exploration and accelerates convergence toward better policies. This is controlled by a shaping factor α, which determines the fraction of failed experiences in the minibatch that will be modified.

Our reward shaping mechanism is loosely inspired by ideas from *Hindsight Experience Replay* (HER) [1] and *Soft HER* [19], where failed trajectories are relabeled with alternate goals to simulate successful episodes. However, unlike HER-based approaches, our method does not assume access to a goal-conditioned environment or rely on relabeling mechanisms tied to domain-specific goal definitions. Table 1 summarizes the conceptual differences between RS-DRL and HER-style experience relabeling.

Instead, RS-DRL employs **randomized reward reshaping**, where a randomly selected subset of failed experiences in the minibatch are assigned a fixed optimistic reward ($r_i \leftarrow 1$). This simulates hypothetical success without requiring any semantic understanding of goals. The core intuition is that failed experiences may still contain valuable exploratory behavior that should not be discarded. By randomly injecting optimistic feedback into these experiences, the agent is

encouraged to revisit and re-evaluate them during learning. This process helps prevent premature convergence to local optima by increasing the diversity of valuable learning signals.

The use of a constant reward of 1 is both deliberate and domain-agnostic. Rather than relying on handcrafted shaping heuristics or scaled rewards, we use a uniform optimistic signal to bias the value function toward considering alternate action paths that would otherwise be ignored due to consistently negative or zero rewards. This simple yet effective strategy promotes generalization and exploration, particularly in environments like self-adaptive systems, where goals are implicit, emergent, or dynamically shifting.

Table 1. Comparison between RS-DRL and Hindsight Experience Replay methods

Method	**Goal Required**	**Domain Knowledge**	**Reward Strategy**	**Target Domain**
HER [1]	Yes	Yes	Goal relabeling	Goal-conditioned RL
Soft HER [19]	Yes	Yes	Soft relabeling	Goal-conditioned RL
RS-DRL (ours)	No	No	Randomized optimistic reward ($r_i \leftarrow 1$)	self-adaptive systems

The algorithm takes as input a minibatch of experiences $B = \{(s_i, a_i, r_i, s_{i+1})\}$ and a reshaping factor α. It first computes the number of experiences N in the minibatch (line **1**) and then calculates K, the number of failed experiences to be reshaped, based on α and N (line **2**). The algorithm then identifies all experiences in B with a reward $r_i = 0$, treating them as failures (line **3**).

If the number of such failed experiences is at least K, a random subset of K failures is selected (line **4**), and for each selected failed experience $(s_i, a_i, 0, s_{i+1})$, the reward is reshaped to 1 (lines **5–7**). If there are fewer than K failed experiences, all such failures are reshaped (line **9**). Finally, the updated minibatch is returned (line **10**).

This reward shaping mechanism provides a simple yet effective way to modify the agent's learning trajectory by emphasizing failed experiences that are reinterpreted as successful ones. The α parameter controls how many of these failures are reshaped, allowing for tunable optimism in learning. The stochastic selection of which failures to reshape (when possible) introduces variability, helping to prevent overfitting to specific transitions.

The value of 1 as the optimistic reward in our shaping mechanism is motivated by the domain characteristics of self-adaptive systems. In many such systems, including our case study DeltaIoT, adaptation goals are framed as satisfying certain QoS constraints (e.g., keeping packet loss below a threshold). These goals are naturally represented as binary outcomes: 1 when the objective is satisfied, and 0 otherwise. Thus, assigning $r_i \leftarrow 1$ to failed transitions simulates an optimistic scenario where the QoS requirement was met. This binary success/failure framing aligns with the semantics of reward in many self-adaptive domains, reinforcing the meaningfulness of the reshaping process.

Input: Minibatch of experiences $B = \{(s_i, a_i, r_i, s_{i+1})\}$, reshaping factor ρ
Output: Updated minibatch with reshaped rewards
Function `RewardShaping(`B, ρ`)`**:**

1. $N \leftarrow |B|$; `// Number of experiences in the minibatch`
2. $K \leftarrow \rho \cdot N$; `// Number of experiences to reshape based on` ρ
3. Identify all failed experiences in B where $r_i = 0$; `// Assume binary rewards`
4. if *number of failed experiences* $\geq K$ **then**
  5. Randomly select K failed experiences from B;
  6. for *each selected experience* $(s_i, a_i, 0, s_{i+1})$ **do**
    7. Set $r_i \leftarrow 1$; `// Reshape failure (0) to success (1)`
  end
end
8. else
  9. Reshape all failed experiences in B to 1 ; `// If fewer than` K `failures exist`
end
10. return updated B;

Algorithm 2: Reward Shaping

6 Evaluation

To evaluate the effectiveness and efficiency of RS-DRL, we applied it to the DeltaIoT system [20], a well-established wireless sensor network (WSN) testbed (see Section 3). This case study allowed us to assess how RS-DRL performs in real-world scenarios where adaptation is crucial for maintaining system performance in dynamic environments.

6.1 Implementation Details

For the implementation of RS-DRL, we utilized several tools and frameworks. To create the reinforcement learning (RL) environment for DeltaIoT, we employed the *Gymnasium* [44] library, which provides a flexible and modular framework for developing and testing RL algorithms. This allowed us to accurately model the DeltaIoT environment's states, actions, and rewards within the RL framework.

For training the RS-DRL agent we used the DeltaIoT adaptation-space dataset provided by Gheibi and Weyns [18]. This dataset supplies the adaptation-cycle traces and environmental drift parameters that we used to instantiate the Gymnasium DeltaIoT environment and to generate the runtime states and transitions used during training.

The core of our implementation is based on the Deep Q-Network (DQN) algorithm from the Stable Baselines3 [41] library, a reliable and widely-used resource for reinforcement learning in Python. We extended the DQN implementation in Stable Baselines3 by integrating our novel reward shaping mechanism, a key aspect of RS-DRL. This modification enables the agent to reshape rewards during the experience replay phase, enhancing its ability to generalize and learn from past failures effectively.

This implementation setup enabled us to systematically test and evaluate the performance of RS-DRL within the DeltaIoT environment, providing insights into its ability to handle dynamic and complex adaptation challenges in

real-world settings. Each configuration was evaluated over 30 independent runs with different random seeds. Results are reported as mean ± standard deviation and 95% confidence intervals, where the confidence intervals are computed for the mean across the 30 runs.

6.2 Evaluation Metrics

To comprehensively assess the performance of RS-DRL in terms of effectiveness and efficiency, we use metrics from [43]: *Asymptotic performance*, which reflects the ultimate performance level the agent achieves after sufficient training, measures effectiveness. *Total performance*, which measures overall learning performance by computing the area between the reward curve and the asymptotic reward, is formally defined by equation 3:

$$TP = R_\infty \cdot T - \sum_{t=1}^{T} r_t \,, \tag{3}$$

where r_t denotes the average reward at step t, R_∞ is the asymptotic performance (the mean reward over the last K steps), and T is the training horizon. Positive values indicate that the reward curve remains below its asymptotic level and improves toward it, while negative values indicate the curve initially lies above the asymptotic level but decays or converges to a lower reward. This metric thus assesses effectiveness by evaluating cumulative progress toward the learning goal. *Time to threshold*, which quantifies the number of time steps required to reach a predefined reward threshold, measures efficiency.

Beyond these core metrics, we also examine the impact of RS-DRL on system metrics such as latency, packet loss, and energy consumption. Comparing these effects provides further insights into the trade-offs and benefits of RS-DRL in dynamic environments like DeltaIoT, validating its robustness and applicability in real-world applications.

6.3 System Goals

We consider three types of goals: minimization, threshold, and setpoint. We now explain each one in more details:

(1) **Minimization goals:**
Energy Consumption is a critical factor in wireless sensor networks (WSNs) as the motes are battery-powered. The goal is to minimize the total energy consumed by all motes during data transmission. Let E_i represent the energy consumption of mote i, and N denote the total number of motes in the network. The objective is to minimize the total energy consumption:

$$\text{Minimize } E_{\text{total}} = \sum_{i=1}^{N} E_i \tag{4}$$

Packet loss refers to the percentage of data packets that fail to reach the gateway. High packet loss can significantly degrade the system's performance. Let P_{sent} represent the total number of packets sent by all motes, and P_{received} represent the total number of packets successfully received by the gateway. The packet loss rate, P_{loss}, is given by:

$$\text{Minimize } P_{\text{loss}} = \frac{P_{\text{sent}} - P_{\text{received}}}{P_{\text{sent}}} \tag{5}$$

Latency is the time it takes for a data packet to travel from a sensor to the gateway. High latency can affect real-time monitoring and decision-making processes. Let L_i represent the latency experienced by mote i, and the objective is to minimize the average latency across all motes:

$$\text{Minimize } L_{\text{avg}} = \frac{1}{N} \sum_{i=1}^{N} L_i \tag{6}$$

Multi-Objective Optimization. In practical scenarios, it is often necessary to optimize for all three objectives simultaneously. This can be achieved using a weighted sum approach, where each objective is assigned a weight reflecting its importance. The combined objective function, F, is given by:

$$\text{Minimize } F = w_1 \cdot \frac{E_{\text{total}}}{E_{\text{max}}} + w_2 \cdot \frac{P_{\text{loss}}}{P_{\text{max}}} + w_3 \cdot \frac{L_{\text{avg}}}{L_{\text{max}}} \tag{7}$$

where w_1, w_2, and w_3 are the weights assigned to energy consumption, packet loss, and latency, respectively, such that $w_1 + w_2 + w_3 = 1$. Here, E_{max}, P_{max}, and L_{max} represent the maximum possible values for energy consumption, packet loss, and latency, respectively, to normalize the objectives. By adjusting the weights, different trade-offs between the objectives can be explored.

(2) **Threshold goals**: A threshold goal requires that some quality property should either remain below or above a given value. The satisfaction of a threshold goal $t \in T$ with a threshold value x for any value of the quality property q is defined by equations 8 and 9:

$$t < x(q) = \begin{cases} \text{True} & \text{if } q < x \\ \text{False} & \text{if } q \geq x \end{cases} \tag{8}$$

$$t > x(q) = \begin{cases} \text{True} & \text{if } q > x \\ \text{False} & \text{if } q \leq x \end{cases} \tag{9}$$

(3) **Setpoint goals**: A set-point goal states that some value of a quality property of the system should be kept at a certain value, i.e., the set-point, with a margin of at most ϵ. The satisfaction of a set-point goal $s \in S$ with target μ and error margin ϵ is defined by equation 10:

$$s_{\mu,\epsilon}(q) = \begin{cases} \text{True} & \text{if } \mu - \epsilon \leq q \leq \mu + \epsilon \\ \text{False} & \text{otherwise} \end{cases} \tag{10}$$

Table 2 shows the adaptation goals used for the evaluation of DeltaIoT: TTS (2 Threshold goals and 1 Set-point goal), TT (2 Threshold goals).

6.3.1 RL Formulation for DeltaIoT Case Study. The RS-DRL method is generic and can be applied to various systems. For the evaluation case study (DeltaIoT), we first formulate its adaptation problem in RL terms. This involves defining the state space, action space, and reward function for DeltaIoT.

In formulating the problem of selecting a suitable adaptation option in DeltaIoT in terms of Reinforcement Learning (RL), we need to define the state space, the action space, and the reward function.

State Space: The state space is represented as:

$$S = \{(E, P, L) \mid E \in \mathbb{R}, P \in [0, 100], L \in [0, 100]\}$$

Table 2. Overview of adaptation goals used for the evaluation of DeltaIoTv1 and DeltaIoTv2 [20]. The adaptation goals are defined for packet loss (PL), latency (LA), and energy consumption (EC).

Type	Goals DeltaIoTv1	Goals DeltaIoTv2
TTS	T1: PL < 10% T2: LA < 15% S1: EC in [13.2 ± 0.1]	T1: PL < 10% T2: LA < 15% S1: EC in [67 ± 0.3]
TT	T1: PL < 10% T2: LA < 5%	T1: PL < 15% T2: LA < 10%

where E, a real-valued number, represents the current energy consumption level of the system. P represents the current packet loss rate during data transmission along the links, ranging from 0 to 100. L represents the current latency experienced by packets in the network, ranging from 0 to 100.

Action Space: The action space consists of potential adaptation options aimed at optimizing quality properties. For example, in DeltaIoTv1, we have 216 adaptation options. The action space consists of these 216 discrete actions. Each action represents a specific adaptation option that can be taken to optimize the performance of the DeltaIoT system. These adaptation options may include adjustments to mote transmission power, packet routing configurations, or other parameters relevant to energy consumption, packet loss, and latency reduction. For example, each action could be encoded as a unique identifier corresponding to a particular combination of parameters that define the adaptation option. In summary, the action space consists of 216 distinct actions, each representing a specific adaptation option aimed at optimizing system performance in terms of energy consumption, packet loss, and latency. Let $A = \{a_1, a_2, \ldots, a_n\}$ denote the action space where a_i represents the i-th adaptation option.

Each adaptation option a_i is described by a combination of parameters denoted by p_{ij}, where $j = 1, 2, \ldots, n_i$ and n_i represents the number of parameters associated with adaptation option a_i. Each p_{ij} can take on specific values denoted by v_{ijk}, where $k = 1, 2, \ldots, m_{ij}$ and m_{ij} represents the number of possible values for p_{ij}.

For example, in DeltaIoTv1, there are 216 adaptation options, so $\|A\| = 216$. Each adaptation option consists of a combination of parameters that can be adjusted to optimize system performance. Each adaptation option a_i can be encoded as a unique identifier corresponding to its specific combination of parameters.

Reward Function: The reward function is designed to optimize multiple objectives, including energy consumption, packet loss, and latency. Depending on the goal, we either normalize these metrics or use their raw values directly to compute the reward. The reward is formulated as follows:

For goals that involve normalized metrics (such as minimizing energy consumption, packet loss, and latency simultaneously), the reward is calculated using:

$$R = \left(\frac{E_{\max} - E}{E_{\max} - E_{\min}}\right) \cdot c_E + \left(\frac{P_{\max} - P}{P_{\max} - P_{\min}}\right) \cdot c_P + \left(\frac{L_{\max} - L}{L_{\max} - L_{\min}}\right) \cdot c_L \tag{11}$$

where E represents the energy consumption, P is the packet loss, and L is the latency. The terms $E_{\max}, E_{\min}, P_{\max}, P_{\min}, L_{\max}, L_{\min}$ denote the maximum and minimum values for energy, packet loss, and latency, respectively. The coefficients c_E, c_P, c_L are used to assign relative importance to energy consumption, packet loss, and latency in the reward calculation.

This reward function allows for flexible optimization across different performance metrics by adjusting the coefficients according to the desired goal, such as focusing on energy efficiency, packet loss reduction, or latency improvement.

Handling Single-Objective Cases In scenarios where only one of the metrics—energy consumption, packet loss, or latency—is the primary concern, the reward function can be adapted to focus exclusively on that objective. This is achieved by setting the coefficients of the other metrics to zero. Below are examples of how the reward function is tailored for single-objective optimization.

Energy-Only Optimization When the goal is to minimize energy consumption alone, the reward function is simplified by setting the coefficients for packet loss (c_P) and latency (c_L) to zero, thus focusing solely on the energy term:

$$R = \left(\frac{E_{\max} - E}{E_{\max} - E_{\min}} \right) \cdot c_E \tag{12}$$

In this case, the system is optimized purely for energy efficiency, and any trade-offs in packet loss or latency are ignored.

In summary, the reward function is flexible enough to handle single-objective cases by adjusting the coefficients of non-relevant objectives to zero, thus focusing the optimization exclusively on the desired metric. This allows the system to be tailored for specific performance goals such as energy efficiency, packet loss reduction, or latency improvement, depending on the use case.

6.3.2 Training DeltaIoT. To begin training the DRL model for the DeltaIoT case, we must set up our learning process by defining the number of timesteps per episode and the overall training horizon. All training episodes use the DeltaIoT traces from Gheibi and Weyns [18], which provide the interference and message-load sequences used to generate the adaptation cycles.[1]

Each DeltaIoT instance corresponds to a different number of adaptation options per cycle: DeltaIoTv1 has 216 options, DeltaIoTv1.1 has 1,296 options, and DeltaIoTv2 has 4,096 options. We treat successive adaptation cycles as a continuous stream rather than independent episodes. Accordingly, we set the training horizons as follows: for DeltaIoTv1, 10,000 timesteps (i.e., enough concatenated 216-step cycles to reach the horizon); for DeltaIoTv1.1, 64,800 timesteps (concatenated 1,296-step cycles); and for DeltaIoTv2, sequences of 4,096-step cycles with the same principle. Training and validation splits are applied at the timestep level (e.g., 1,000 timesteps for training and 200 timesteps for validation), preserving temporal continuity across cycles and better matching a streaming runtime environment.

At the start of each cycle, energy consumption, packet loss, and latency—objectives that require continuous monitoring in DeltaIoT—are collected from the environment and passed to the feature extractor. The feature extractor converts them into a state representation and passes them to the Adaptation Option Selector, where an adaptation option is selected and then passed to the Adaptation Option Verifier. The Adaptation Option Verifier, which contains a model of the system and the environment, computes the corresponding energy consumption, packet loss, and latency of the selected adaptation option, i.e., the next state, and invokes the Reward Calculator. Based on the defined goals of the system, the Reward Calculator computes a reward. Subsequently, the state, selected adaptation option, reward, and next state are stored in the Replay Memory.

To initiate the training process, the Replay Memory must be filled up to a certain threshold. Therefore, the Model Training component checks if the Replay Memory has enough data. If so, it extracts a batch of data from the Replay Memory and begins training. The trained model is then passed to the Adaptation Option Selector. This iterative process continues until the specified training cycle time elapses. Additionally, an evaluation phase is specified where, every n training cycles, the training is paused, and the learned model is evaluated for m cycles to assess its predictive power. If

[1]The DeltaIoT datasets are publicly available at the LLSAS project website: https://people.cs.kuleuven.be/~danny.weyns/software/LLSAS/.

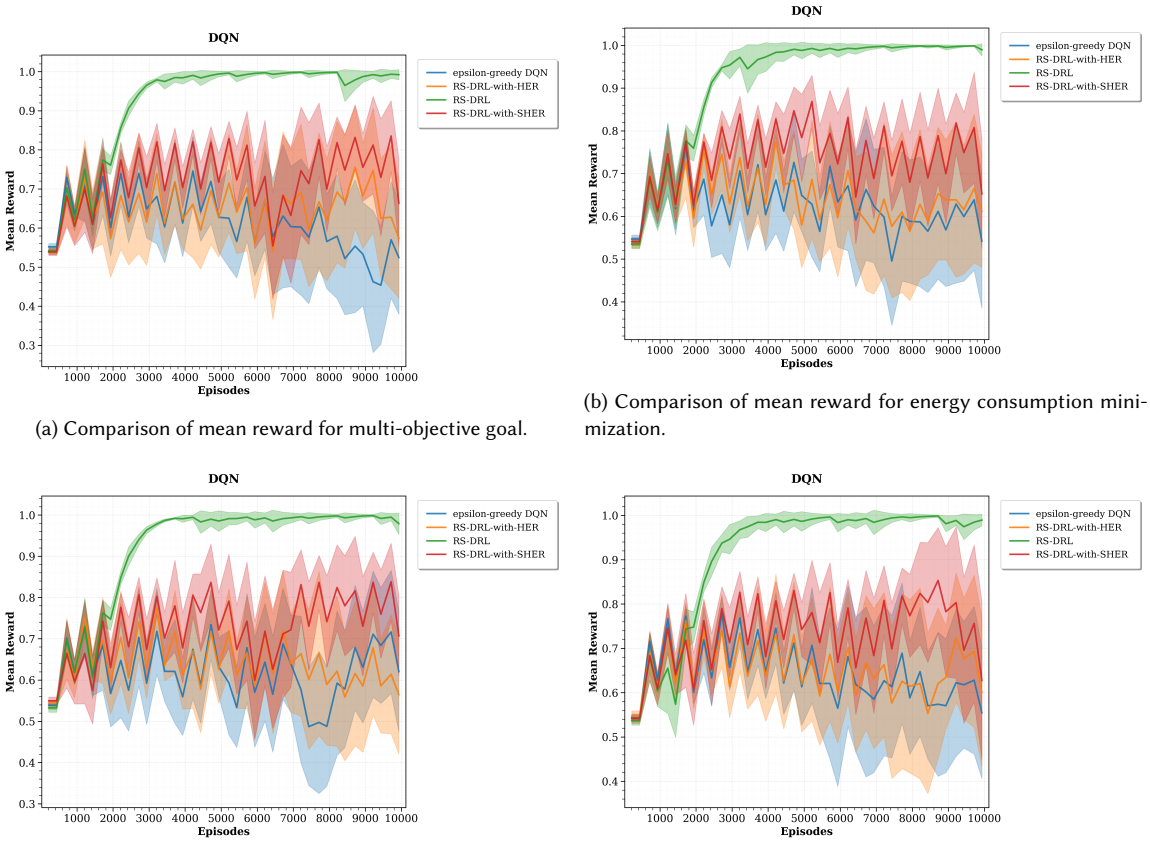
the new model outperforms the previous one in terms of quality properties, it is stored in the Knowledge Repository of the MAPE-K and replaces the old model.

6.4 Hyperparameter Optimization

The selection of the most suitable hyperparameters was conducted through a systematic grid search approach combined with empirical evaluation. Grid search involves exhaustively searching through a predefined set of hyperparameter values and evaluating the performance of the model for each combination. This process is computationally intensive but ensures a comprehensive exploration of the hyperparameter space.

During the grid search, the model's performance was assessed using a validation set, focusing on key metrics such as convergence speed, stability, and the quality of the selected adaptation options. The combination of hyperparameters that yielded the best trade-off between these metrics was selected as the optimal configuration for the RS-DRL model.

6.5 Comparison with other methods



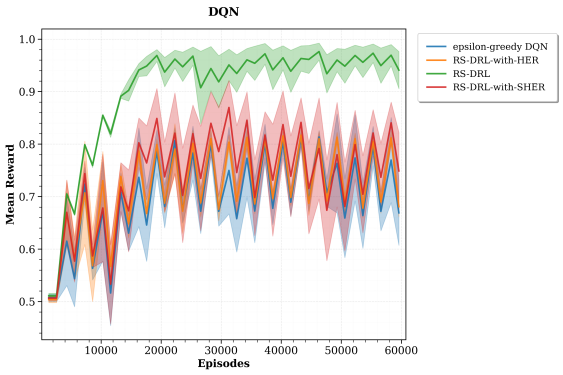
(a) Comparison of mean reward for multi-objective goal.

(b) Comparison of mean reward for energy consumption minimization.

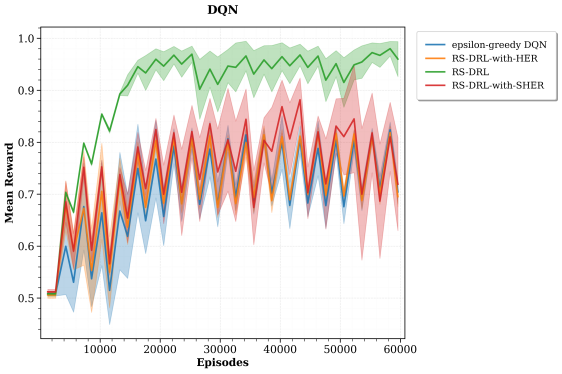
(c) Comparison of mean reward for packet loss minimization.

(d) Comparison of mean reward for latency minimization.

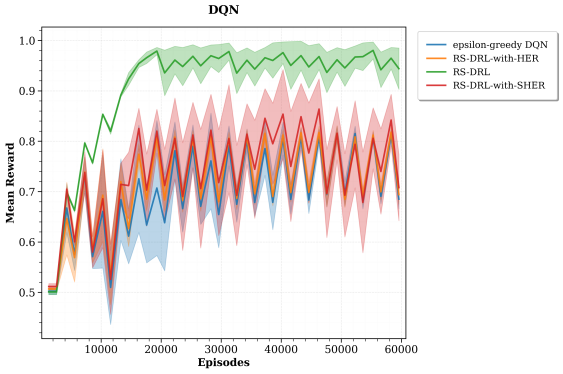
Fig. 5. Comparison of mean rewards among RS-DRL, DQN (with epsilon-greedy strategy), RS-DRL-with-HER, and RS-DRL-with-SHER over 10,000 steps for various objectives in DeltaIoTv1



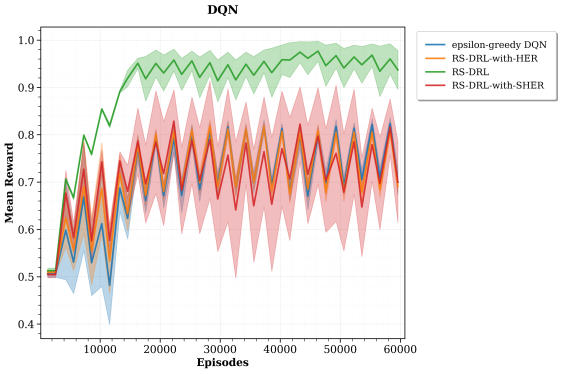
(a) Comparison of mean reward for multi-objective goal.



(b) Comparison of mean reward for energy consumption minimization.



(c) Comparison of mean reward for packet loss minimization.



(d) Comparison of mean reward for latency minimization.

Fig. 6. Comparison of mean rewards among RS-DRL, DQN (with epsilon-greedy strategy), RS-DRL-with-HER, and RS-DRL-with-SHER over 64,800 steps for various objectives in DeltaIoTv1.1

We compare RS-DRL with the following methods:

a) For minimization goals (energy consumption, packet loss, and latency), we compare RS-DRL against (i) an epsilon-greedy exploration strategy (see Section 2.3), and (ii) reward shaping variants of our method that incorporate *Hindsight Experience Replay (HER)* and *Soft Hindsight Experience Replay (SHER)*, referred to hereafter as **RS-DRL-with-HER** and **RS-DRL-with-SHER**. This comparison highlights how different exploration strategies and reward shaping techniques affect convergence performance. Results are reported with standard deviations to provide confidence intervals and assess robustness. The outcomes are illustrated in Figures 5 and 6.

b) For the TTS goal, we used a competing method DLASeR+ [50] that applies a deep neural network to reduce adaptation spaces.

c) For the TT goal, we used three competing methods, namely DLASER [45], DLASER+ [50], and ML4EAS [39], which use supervised machine learning approaches.

Also for both TTS and TT goals, we used a Reference method that exhaustively analyzes the entire adaptation space without employing machine learning. This exhaustive-search baseline evaluates all possible adaptation options using

runtime model checking to identify the best configuration. While it represents an ideal upper bound on quality, its reported averages may occasionally appear slightly worse in some tables due to runtime noise and stochastic evaluation. We follow the convention in [39, 50] in referring to this exhaustive analysis as the Reference method.

6.5.1 Limitations of Baseline Comparisons. While HER and Soft HER are designed for goal-conditioned environments, we adapt them to DeltaIoT by treating QoS thresholds (Table 2) as proxy goals for relabeling. Specifically:

- **Goal Proxy**: Trajectories are relabeled as successful if they retrospectively meet threshold goals (e.g., packet loss <10%), even if they failed for their original objective.
- **Reward Redefinition**: HER's binary goal rewards are replaced with DeltaIoT's multi-objective reward function (Eq. 11) to maintain consistency with RS-DRL's optimization framework.

Example: For RS-DRL-with-HER, a trajectory initially failing to minimize energy consumption is relabeled as successful if it achieves packet loss <10%, with rewards recalculated using Eq. 11. Despite these adaptations, RS-DRL-HER/Soft HER underperform RS-DRL (Tables 3–4), highlighting their inherent reliance on explicit goal definitions—a limitation RS-DRL avoids through randomized reward shaping (Algorithm 2).

Table 3. Comparison of Different Methods on Various Goals Using Key Performance Metrics for DeltaIoTv1. The values are reported as mean ± standard deviation.

Comparison of Methods for Different Goals (DeltaIoTv1)				
Objective	**Method**	**Asymptotic Perf.**	**Time to Threshold (0.9)**	**Total Perf.**
Multi-objective	RS-DRL	**0.9905 ± 0.0268**	**2266 ± 150**	**2.2346 ± 1.1786**
	RS-DRL-with-HER	0.4927 ± 0.1714	does not reach threshold	-6.3227 ± 6.4043
	RS-DRL-with-SHER	0.4056 ± 0.2331	6216 ± 1080	-12.8771 ± 8.8921
	ϵ-greedy DQN	0.4559 ± 0.2233	does not reach threshold	-6.4589 ± 9.0992
Energy Consumption	RS-DRL	**0.9756 ± 0.0545**	**2316 ± 300**	**1.7036 ± 2.3210**
	RS-DRL-with-HER	0.5337 ± 0.1081	does not reach threshold	-4.4936 ± 4.0335
	RS-DRL-with-SHER	0.4197 ± 0.2028	4216 ± 0	-12.8512 ± 8.6149
	ϵ-greedy DQN	0.3976 ± 0.2338	does not reach threshold	-9.1468 ± 8.7002
Latency	RS-DRL	**0.9964 ± 0.0107**	**2516 ± 509**	**2.7962 ± 0.7629**
	RS-DRL-with-HER	0.4621 ± 0.2649	7216 ± 0	-7.6636 ± 10.4890
	RS-DRL-with-SHER	0.4348 ± 0.2898	7591 ± 819	-12.0126 ± 11.4374
	ϵ-greedy DQN	0.4444 ± 0.1947	does not reach threshold	-8.0365 ± 7.5503
Packet Loss	RS-DRL	**0.9850 ± 0.0071**	**2366 ± 320**	**2.7142 ± 0.4937**
	RS-DRL-with-HER	0.4900 ± 0.2310	does not reach threshold	-6.0904 ± 9.5651
	RS-DRL-with-SHER	0.5104 ± 0.1628	4466 ± 2250	-8.8320 ± 6.4889
	ϵ-greedy DQN	0.4763 ± 0.2295	does not reach threshold	-5.8930 ± 8.4972

6.5.2 Minimization Goals. Tables 3 and 4 summarize the performance of RS-DRL and baseline methods across multiple objectives in the DeltaIoTv1 and DeltaIoTv1.1 scenarios. The corresponding learning curves are shown in Figures 5 and 6.

In the multi-objective scenario, RS-DRL consistently achieves the highest asymptotic performance and fastest convergence. On DeltaIoTv1, it reaches a mean reward of **0.9905 ± 0.0268** in **2266 ± 150** steps, with a total performance of **2.2346 ± 1.1786**. In comparison, RS-DRL-with-HER, RS-DRL-with-SHER, and ϵ-greedy DQN either fail to reach

Table 4. Comparison of Different Methods on Various Goals Using Key Performance Metrics for DeltaIoTv1.1 The values are reported as mean ± standard deviation.

Comparison of Methods for Different Goals (DeltaIoTv1.1)				
Objective	**Method**	**Asymptotic Perf.**	**Time to Threshold (0.9)**	**Total Perf.**
Multi-objective	RS-DRL	**0.8983 ± 0.0932**	**19296 ± 10099**	**-0.6906 ± 3.8895**
	RS-DRL-with-HER	0.4843 ± 0.0460	does not reach threshold	-9.4843 ± 1.7298
	RS-DRL-with-SHER	0.6114 ± 0.2004	13296 ± 0	-5.3747 ± 7.9180
	ϵ-greedy DQN	0.5180 ± 0.0524	does not reach threshold	-7.3837 ± 2.5238
Energy Consumption	RS-DRL	**0.9301 ± 0.1017**	**18396 ± 5700**	**0.6357 ± 3.9618**
	RS-DRL-with-HER	0.5101 ± 0.0475	does not reach threshold	-8.4646 ± 2.3410
	RS-DRL-with-SHER	0.5826 ± 0.1995	41796 ± 4500	-6.7035 ± 8.1707
	ϵ-greedy DQN	0.5251 ± 0.0546	does not reach threshold	-7.3376 ± 2.3111
Latency	RS-DRL	**0.9021 ± 0.0877**	**18696 ± 5969**	**-0.3886 ± 3.4555**
	RS-DRL-with-HER	0.5145 ± 0.0384	does not reach threshold	-8.0655 ± 1.3917
	RS-DRL-with-SHER	0.5253 ± 0.2039	34296 ± 0	-7.6619 ± 7.9208
	ϵ-greedy DQN	0.5128 ± 0.0784	does not reach threshold	-7.9400 ± 3.3884
Packet Loss	RS-DRL	**0.9129 ± 0.1136**	**13596 ± 900**	**-0.3138 ± 4.5171**
	RS-DRL-with-HER	0.5085 ± 0.0515	does not reach threshold	-8.5602 ± 1.7917
	RS-DRL-with-SHER	0.5068 ± 0.1411	31296 ± 9000	-9.4814 ± 5.1290
	ϵ-greedy DQN	0.4941 ± 0.0547	does not reach threshold	-8.4545 ± 2.2614

the performance threshold or exhibit considerably lower total rewards. On the more complex DeltaIoTv1.1, RS-DRL maintains its advantage, achieving **0.8983 ± 0.0932** in **19296 ± 10099** steps, surpassing all baseline methods in both speed and total reward.

For energy consumption optimization, RS-DRL again demonstrates superior results. On DeltaIoTv1, it achieves **0.9756 ± 0.0545** in **2316 ± 300** steps with total performance **1.7036 ± 2.3210**, while all baselines fail to reach competitive thresholds. On DeltaIoTv1.1, RS-DRL attains **0.9301 ± 0.1017** in **18396 ± 5700** steps, maintaining the highest total performance among all methods.

In the packet loss optimization scenario, RS-DRL achieves near-optimal performance on DeltaIoTv1: **0.9850 ± 0.0071** in **2366 ± 320** steps with total performance **2.7142 ± 0.4937**, far exceeding baseline methods. On DeltaIoTv1.1, it continues to outperform competitors with **0.9129 ± 0.1136** in **13596 ± 900** steps.

For latency optimization, RS-DRL again outperforms all other methods. On DeltaIoTv1, it reaches **0.9964 ± 0.0107** in **2516 ± 509** steps with strong total performance. On DeltaIoTv1.1, RS-DRL achieves **0.9021 ± 0.0877** in **18696 ± 5969** steps, outperforming all baseline approaches in both convergence and total reward.

Overall, across all objectives, RS-DRL consistently provides superior performance in terms of convergence speed, final asymptotic reward, and total performance, demonstrating its effectiveness in complex, resource-constrained IoT environments (Figures 5 and 6).

6.6 Statistical Analysis of RS-DRL Effectiveness

To validate the reported improvements, we conducted statistical significance testing using one-sample t-tests at a significance level of $\alpha = 0.05$ [16]. The one-sample t-test compares the sample mean of RS-DRL to a fixed reference value

Table 5. Statistical Validation of RS-DRL Effectiveness. RS-DRL values are shown as $a \pm b[c, d]$, where a is the mean, b the standard deviation, and $[c, d]$ the 95% bootstrap confidence interval. "–" indicates missing data from original publications.

Goal	Case Study	Method	PL	LA	EC
TTS	DeltaIoTv1	**RS-DRL**	**10.578 ± 0.478 [10.383, 10.766]**	**0.097 ± 0.009 [0.093, 0.101]**	**12.882 ± 0.296 [12.769, 12.992]**
		Reference	14.230	0.100	13.100
		DLASER+ [50]	12.890	0.840	13.020
	DeltaIoTv2	**RS-DRL**	**11.512 ± 0.302 [11.405, 11.622]**	**0.251 ± 0.021 [0.243, 0.259]**	**66.797 ± 1.011 [66.433, 67.177]**
		Reference	12.830	0.100	67.000
		DLASER+ [50]	12.990	0.850	67.010
TT	DeltaIoTv1	**RS-DRL**	**7.153 ± 0.257 [7.057, 7.254]**	**0.170 ± 0.009 [0.167, 0.174]**	-
		Reference	8.720	0.100	-
		ML4EAS [39]	8.620	0.100	-
		DLASER+ [50]	7.980	0.470	-
		DLASER [45]	8.180	0.200	-
	DeltaIoTv2	**RS-DRL**	**9.330 ± 0.487 [9.150, 9.514]**	**1.310 ± 0.095 [1.273, 1.347]**	-
		Reference	11.840	0.100	-
		ML4EAS [39]	10.560	1.250	-
		DLASER+ [50]	11.390	2.010	-
		DLASER [45]	11.350	2.150	-

Table 6. P-values for RS-DRL vs Baseline Comparisons. P-values are from one-sample t-tests comparing RS-DRL to baseline scalars. "–" indicates no data available for comparison.

Goal	Case Study	Metric	Reference	ML4EAS	DLASER	DLASER+
TTS	DeltaIoTv1	PL	< 0.001	–	–	< 0.001
		LA	0.134	–	–	< 0.001
		EC	0.001	–	–	0.028
	DeltaIoTv2	PL	< 0.001	–	–	< 0.001
		LA	< 0.001	–	–	< 0.001
		EC	0.326	–	–	0.303
TT	DeltaIoTv1	PL	< 0.001	< 0.001	< 0.001	< 0.001
		LA	< 0.001	< 0.001	< 0.001	< 0.001
		EC	–	–	–	–
	DeltaIoTv2	PL	< 0.001	< 0.001	< 0.001	< 0.001
		LA	< 0.001	0.004	< 0.001	< 0.001
		EC	–	–	–	–

when baseline methods provide only single scalar results without per-run variance estimates. For each comparison, 30 independent RS-DRL runs (different random seeds) were used.

The hypotheses for each performance metric (PL, LA, EC) were defined as follows, assuming the objective of minimizing all metrics. The **null hypothesis** (H_0) states that $\mu_{RS\text{-}DRL} \geq \mu_{baseline}$, while the **alternative hypothesis** (H_1) states that $\mu_{RS\text{-}DRL} < \mu_{baseline}$. Here, $\mu_{RS\text{-}DRL}$ denotes the RS-DRL mean and $\mu_{baseline}$ the baseline scalar value. The t-statistic is computed by equation 13:

$$t = \frac{\bar{x} - \mu_0}{\frac{s}{\sqrt{n}}}, \tag{13}$$

with $\bar{x}$, s, and n denoting the sample mean, standard deviation, and sample size of RS-DRL runs, and μ_0 the baseline scalar value. One-tailed p-values were used to test whether RS-DRL achieved significantly *lower* values than each baseline.

A total of 28 statistical comparisons were performed. For the **TTS goal**, there are 6 rows and 2 baselines (Reference and DLASER+ [50]), resulting in 12 comparisons. For the **TT goal**, there are 8 rows and 4 baselines (Reference, ML4EAS [39], DLASER [45], and DLASER+ [50]), resulting in 16 comparisons. Together, these yield 28 valid comparisons after excluding missing "–" entries.

The results in Tables 5 and 6 show that RS-DRL achieved statistically significant improvements in 25 of the 28 comparisons (89.3%). For **Packet Loss (PL)**, RS-DRL significantly reduced PL in *all* comparisons (100% significance). Regarding **Latency (LA)**, for the TTS goal in **DeltaIoTv1**, RS-DRL achieved latency comparable to the Reference baseline (0.097 vs. 0.100, $p = 0.134$), while substantially outperforming DLASER+ ($p < 0.001$). For **DeltaIoTv2**, latency was *higher* than Reference (0.251 vs. 0.100, $p < 0.001$), reflecting a throughput–latency trade-off rather than an improvement. Under the TT goal, RS-DRL again showed *higher latency* in both case studies, consistent with the objective prioritizing throughput over latency. Finally, for **Energy Consumption (EC)**, RS-DRL achieved a small but significant improvement in **DeltaIoTv1** TTS (1.7%, $p = 0.001$) and no significant difference in **DeltaIoTv2** TTS ($p = 0.326$). EC metrics were not reported for TT by any baseline.

Overall, the combination of one-sample t-tests, p-values, and 95% confidence intervals consistently confirms that RS-DRL provides statistically and practically significant gains in packet loss reduction and energy efficiency, while revealing expected latency trade-offs under throughput-focused optimization.

6.7 Analysis and Discussion

One of the most critical strengths of RS-DRL, as shown in Figures 5 and 6, lies in its ability to escape local optima—a common challenge in DRL systems [14]. The mean reward for RS-DRL not only increases steadily but also stabilizes at a higher level compared to the epsilon-greedy DQN approach, which suffers from significant fluctuations and struggles to escape suboptimal solutions. The stability and faster convergence of RS-DRL are crucial in dynamic environments where system conditions can change rapidly. By integrating reward shaping mechanism, RS-DRL ensures that the model can generalize better across varying scenarios, unlike conventional DRL models that often require retraining to handle changes effectively [9, 52]. This characteristic makes RS-DRL particularly suited for real-time systems, such as cloud computing or IoT, where constant optimization is necessary.

Theoretical Advantages over Goal-Conditioned Methods. While HER-based methods [1] excel in goal-conditioned environments (e.g., robotics), their reliance on static goal definitions limits applicability in self-adaptive systems like DeltaIoT, where QoS thresholds (Table 2) may shift unpredictably due to network interference or load variability. RS-DRL's randomized reward shaping (Algorithm 2) circumvents this limitation by requiring no predefined goals, instead treating exploration as a goal-agnostic process. This aligns with recent advances in non-stationary RL [53], where adaptability to emergent objectives is critical. HER's inferior performance (Tables 3–4) despite our adaptations (Section 6.5.1) underscores RS-DRL's theoretical superiority in dynamic settings.

Compared to other state-of-the-art methods, such as DLASER+ [50] and ML-based methods like ML4EAS [39], RS-DRL demonstrates several distinct advantages. Unlike these approaches, which often require offline labeled training data to perform effectively, RS-DRL leverages a RL-based approach, allowing it to operate without the need for pre-labeled datasets. This distinction not only reduces the overhead associated with data labeling but also enables RS-DRL to adapt

dynamically to changing system conditions in real-time. As shown in Tables 5 and 6, RS-DRL's effectiveness in selecting suitable adaptation options further validates its robustness.

7 Future Work

While RS-DRL demonstrates clear advantages in improving convergence and escaping local optima across the DeltaIoT benchmarks, several aspects warrant further attention. The approach still depends on careful selection of hyperparameters such as the reshaping factor ρ, and convergence times may vary with the size and stochasticity of the adaptation space. Moreover, RS-DRL assumes a discrete, episodic reward formulation, which may limit direct applicability to continuous or safety-critical domains. Addressing these limitations will enhance the robustness and practicality of the approach.

As future work, we plan to optimize the convergence time of RS-DRL and develop adaptive or self-tuning reshaping strategies to minimize manual configuration, enhancing its applicability in real-time systems. Additionally, we aim to explore alternative reward structures and cross-domain validation in other self-adaptive contexts—such as cloud autoscaling, autonomous systems, and cyber-physical environments—to assess generalizability and scalability. Finally, we will investigate ways to reduce computational complexity and analyze theoretical properties related to stability and robustness, further strengthening RS-DRL as a reliable mechanism for managing uncertainty in self-adaptive systems.

8 Conclusion

The article presents a novel method for managing uncertainty in self-adaptive systems using RS-DRL, a Reward Shaping method integrated with Deep Reinforcement Learning (DRL). The proposed method effectively enhances the adaptability and robustness of self-adaptive systems, particularly in dynamic and complex environments like IoT networks. The evaluation results demonstrate that RS-DRL consistently outperforms traditional methods such as the epsilon-greedy DQN. It achieves higher asymptotic performance, faster convergence to optimal policies, and better overall efficiency, making it a promising tool for optimizing self-adaptive systems in real-world applications.

While RS-DRL shows great potential, challenges remain in optimizing its convergence time and computational complexity. Addressing these challenges in future research could further enhance the method's applicability in real-time and resource-constrained environments.

In conclusion, RS-DRL offers a robust solution for managing uncertainty and improving system performance in dynamic and uncertain environments. Its successful application in various scenarios underscores its potential for broader use in other domains requiring adaptive and resilient system behaviors.

References

[1] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. 2017. Hindsight experience replay. In *Advances in Neural Information Processing Systems*, Vol. 30.

[2] Hadi Arabnejad et al. 2017. A comparison of reinforcement learning techniques for fuzzy cloud auto-scaling. In *2017 17th IEEE/ACM international symposium on cluster, cloud and grid computing (CCGRID)*. IEEE.

[3] Eoin Barrett, Enda Howley, and Jim Duggan. 2013. Applying reinforcement learning towards automating resource allocation and application scalability in the cloud. *Concurrency and computation: practice and experience* 25, 12 (2013), 1656–1674.

[4] Radu Calinescu et al. 2010. Dynamic QoS management and optimization in service-based systems. *IEEE Transactions on Software Engineering* 37, 3 (2010), 387–409.

[5] Javier Cámara, Henry Muccini, and Kishore Vaidhyanathan. 2020. Quantitative verification-aided machine learning: A tandem approach for architecting self-adaptive IoT systems. In *2020 IEEE International Conference on Software Architecture (ICSA)*. IEEE.

[6] Mauro Caporuscio et al. 2016. Reinforcement learning techniques for decentralized self-adaptive service assembly. In *Service-Oriented and Cloud Computing: 5th IFIP WG 2.14 European Conference, ESOCC 2016, Vienna, Austria, September 5-7, 2016, Proceedings 5*. Springer.

[7] Leonardo Castañeda, Norha M Villegas, and Hausi A Müller. 2014. Self-adaptive applications: On the development of personalized web-tasking systems. In *Proceedings of the 9th international symposium on software engineering for adaptive and self-managing systems*.

[8] Tao Chen et al. 2018. FEMOSAA: Feature-guided and knee-driven multi-objective optimization for self-adaptive software. *ACM Transactions on Software Engineering and Methodology* 27, 2 (2018).

[9] Karl Cobbe, Oleg Klimov, Christopher Hesse, Tae Kim, and John Schulman. 2020. Quantifying Generalization in Reinforcement Learning. *arXiv preprint arXiv:1812.02341* (2020).

[10] Zachary Coker, David Garlan, and Claire Le Goues. 2015. SASS: Self-Adaptation Using Stochastic Search. In *10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, SEAMS 2015*. Institute of Electrical and Electronics Engineers Inc.

[11] Alexandre David et al. 2015. Uppaal SMC tutorial. *International journal on software tools for technology transfer* 17 (2015), 397–415.

[12] N. De La Fuente and D.A.V. Guerra. 2024. A Comparative Study of Deep Reinforcement Learning Models: DQN vs PPO vs A2C. *arXiv preprint arXiv:2407.14151* (2024).

[13] Rogério De Lemos et al. 2017. Software engineering for self-adaptive systems: Research challenges in the provision of assurances. In *Software Engineering for Self-Adaptive Systems III. Assurances: International Seminar, Dagstuhl Castle, Germany, December 15-19, 2013, Revised Selected and Invited Papers*. Springer.

[14] G. Du, Y. Wang, Y. Liu, X. Wu, and L. Ma. 2023. Alleviating Local Optima and Enhancing Path Planning: A Deep Reinforcement Learning Approach for Autonomous Exploration. In *International Conference on Autonomous Unmanned Systems*. Springer Nature Singapore, Singapore, 124–133.

[15] George Edwards et al. 2009. Architecture-driven self-adaptation and self-management in robotics systems. In *2009 ICSE Workshop on Software Engineering for Adaptive and Self-Managing Systems*. IEEE.

[16] Andy P. Field. 2017. *Discovering Statistics Using IBM SPSS Statistics* (5 ed.). Sage Publications, London.

[17] David Garlan, Shang-Wen Cheng, An Huang, Bradley Schmerl, and Peter Steenkiste. 2004. Rainbow: Architecture-Based Self-Adaptation with Reusable Infrastructure. *Computer* 37, 10 (2004), 46–54. https://doi.org/10.1109/MC.2004.175

[18] Omid Gheibi and Danny Weyns. 2024. Dealing with drift of adaptation spaces in learning-based self-adaptive systems using lifelong self-adaptation. *ACM Transactions on Autonomous and Adaptive Systems* 19, 1 (2024), 1–57.

[19] Qiyang He, Li Zhuang, and Hao Li. 2020. Soft hindsight experience replay. *arXiv preprint arXiv:2002.02089* (2020).

[20] Muhammad Umer Iftikhar et al. 2017. DeltaIoT: A self-adaptive internet of things exemplar. In *2017 IEEE/ACM 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. IEEE.

[21] Pooyan Jamshidi et al. 2016. Fuzzy self-learning controllers for elasticity management in dynamic cloud architectures. In *2016 12th International ACM SIGSOFT Conference on Quality of Software Architectures (QoSA)*. IEEE.

[22] Pooyan Jamshidi et al. 2019. Machine learning meets quantitative planning: Enabling self-adaptation in autonomous robots. In *2019 IEEE/ACM 14th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. IEEE.

[23] Fadwa Kachi and Chafia Bouanaka. 2023. A hybrid model for efficient decision-making in self-adaptive systems. *Information and Software Technology* 153 (2023), 107063.

[24] Jeffrey O Kephart and David M Chess. 2003. The vision of autonomic computing. *Computer* 36, 1 (2003), 41–50.

[25] Donghwan Kim and Sooyong Park. 2009. Reinforcement learning-based dynamic adaptation planning method for architecture-based self-managed software. In *2009 ICSE Workshop on Software Engineering for Adaptive and Self-Managing Systems*. IEEE.

[26] Chase Kinneer et al. 2018. Managing uncertainty in self-adaptive systems with plan reuse and stochastic search. In *ACM/IEEE 13th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, SEAMS 2018, , co-located with International Conference on Software Engineering, ICSE 2018*. IEEE Computer Society.

[27] Chase Kinneer et al. 2020. Building reusable repertoires for stochastic self-* planners. In *2020 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*. IEEE.

[28] Chase Kinneer, David Garlan, and Claire Le Goues. 2021. Information reuse and stochastic search: Managing uncertainty in self-* systems. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* 15, 1 (2021), 1–36.

[29] Christian Krupitzer et al. 2015. A survey on engineering approaches for self-adaptive systems. *Pervasive and Mobile Computing* 17 (2015), 184–206.

[30] Marta Kwiatkowska, Gethin Norman, and David Parker. 2002. PRISM: Probabilistic symbolic model checker. In *International Conference on Modelling Techniques and Tools for Computer Performance Evaluation*. Springer.

[31] Andreas Metzger et al. 2022. Realizing self-adaptive systems via online reinforcement learning and feature-model-guided exploration. *Computing* (2022), 1–22.

[32] Volodymyr Mnih et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533.

[33] P Read Montague. 1999. Reinforcement learning: an introduction, by Sutton, RS and Barto, AG. *Trends in cognitive sciences* 3, 9 (1999), 360.

[34] Gabriel A Moreno et al. 2015. Proactive self-adaptation under uncertainty: a probabilistic model checking approach. In *Proceedings of the 2015 10th joint meeting on foundations of software engineering*.

[35] Ahmed Moustafa and M Zhang. 2014. Learning efficient compositions for QoS-aware service provisioning. In *2014 IEEE International Conference on Web Services*. IEEE.

[36] Henry Muccini, Mona Sharaf, and Danny Weyns. 2016. Self-adaptation for cyber-physical systems: a systematic literature review. In *Proceedings of the 11th international symposium on software engineering for adaptive and self-managing systems*.

[37] Charles Packer, Katelyn Gao, Jeremy Kos, Philipp Krähenbühl, Dawn Song, and Vladlen Koltun. 2019. Assessing Generalization in Deep Reinforcement Learning. *arXiv preprint arXiv:1810.12282* (2019).

[38] G Georgina Pascual, Manuel Pinto, and Lidia Fuentes. 2013. Run-time adaptation of mobile applications using genetic algorithms. In *2013 8th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. IEEE.

[39] Feng Quin et al. 2019. Efficient analysis of large adaptation spaces in self-adaptive systems using machine learning. In *2019 IEEE/ACM 14th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. IEEE.

[40] Feng Quin, Danny Weyns, and Omid Gheibi. 2022. Reducing large adaptation spaces in self-adaptive systems using machine learning. *Journal of Systems and Software* (2022), 111341.

[41] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* 22, 268 (2021), 1–8. http://jmlr.org/papers/v22/20-1364.html

[42] Ross Shaw, Enda Howley, and Eoin Barrett. 2022. Applying reinforcement learning towards automating energy efficient virtual machine consolidation in cloud data centers. *Information Systems* 107 (2022), 101722.

[43] Matthew E. Taylor and Peter Stone. 2009. Transfer learning for reinforcement learning domains: a survey. *Journal of Machine Learning Research* 10 (2009), 1633–1685.

[44] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. 2024. Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032 [cs.LG] https://arxiv.org/abs/2407.17032

[45] Jörg Van Der Donckt et al. 2020. Applying deep learning to reduce large adaptation spaces of self-adaptive systems with multiple types of goals. In *Proceedings of the IEEE/ACM 15th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*.

[46] Danny Weyns. 2017. Software engineering of self-adaptive systems: an organised tour and future challenges. In *Handbook of Software Engineering*. 2.

[47] Danny Weyns. 2020. *An introduction to self-adaptive systems: A contemporary software engineering perspective*. John Wiley & Sons.

[48] Danny Weyns et al. 2017. Perpetual assurances for self-adaptive systems. In *Software Engineering for Self-Adaptive Systems III. Assurances: International Seminar, Dagstuhl Castle, Germany, December 15-19, 2013, Revised Selected and Invited Papers*. Springer.

[49] Danny Weyns et al. 2018. Applying architecture-based adaptation to automate the management of internet-of-things. In *Software Architecture: 12th European Conference on Software Architecture, ECSA 2018, Madrid, Spain, September 24–28, 2018, Proceedings 12*. Springer.

[50] Danny Weyns et al. 2022. Deep learning for effective and efficient reduction of large adaptation spaces in self-adaptive systems. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* 17, 1-2 (2022), 1–42.

[51] Danny Weyns and Muhammad Umer Iftikhar. 2023. ActivFORMS: A formally founded model-based approach to engineer self-adaptive systems. *ACM Transactions on Software Engineering and Methodology* 32, 1 (2023), 1–48.

[52] Jianyu Zhang and Yann LeCun. 2018. A Study on Overfitting in Deep Reinforcement Learning. *arXiv preprint arXiv:1804.06893* (2018).

[53] Tao Zhao et al. 2017. A reinforcement learning-based framework for the generation and evolution of adaptation rules. In *2017 IEEE International Conference on Autonomic Computing (ICAC)*. IEEE.

Author Biographies

Alireza Kavianifar is a Ph.D. candidate in Computer Engineering at Tarbiat Modares University, Tehran, Iran. His research interests include self-adaptive systems, deep reinforcement learning, and uncertainty management in autonomous systems.

Saeed Jalili is a Professor in the Computer Engineering Department at Tarbiat Modares University, Tehran, Iran. His research interests span software engineering, self-adaptive systems, and formal methods. He is the corresponding author of this paper.